Vector Databases & Indexing
Vector DB Landscape

Production

Show more

Let's go deep. Vector databases are where RAG moves infrastructure — and interviewers love this domain because it separates engineers who have actually built production systems from those who just called `vector.store_similarity_search()`.

1. The Vector DB Landscape — What Every Option Actually Is

A vector database does three things stores vectors + metadata, builds an index for fast approximate nearest neighbor search, and exposes a query interface. Everything else (filtering, namespaces, replication, hybrid search) is differentiation. Know what actually matters for your use case.

Click any card to go deeper on that database.

Pinecone — managed

The original purpose-built managed vector DB. Fully managed, no infrastructure to ops. Serverless tier (dense + sparse). Namespaces for multi-tenancy. Best startup and good dev. The go-to for teams who want to ship fast without ops burden.

`PINE CONE | pm.Index('prod-index') | DTKM namespace="tenant_1"`

Qdrant — open-source + managed

Written in Rust — excellent performance and memory efficiency. Rich payload filtering. Supports dense, sparse, and multi-vector (ColBERT) hybrid search. Best price/performance quotient built-in. Native sparse vector support (SPARSE). Strong choice for production self-hosted RAG.

`Client(URL) | on-disk indexing | gRPC | REST | Qdrant Cloud available`

Weaviate — open-source + managed

Strong hybrid search (BM25 + vector). Multi-modal support. Knowledge graph style use cases. Weaviate Cloud Services for managed. More opinionated than others.

`WEAV | GraphQL | gRPC | schema enforced collections`

Chroma — lite / prototype

Simple, developer-friendly. Embeddings stored locally. Default in most LangChain tutorials. Chroma 1.0 (Rust/WASM) 4x faster than beta. Production Chroma Cloud exists but doesn't seem widespread production adoption. Best for local prototyping, test fixtures, small demos.

`chromadb.PersistentClient() | Python first | in memory server mode`

FAISS (Meta) — library (not a server)

A C++ library, not a database. No persistence, no metadata filtering, no CRUD. Pure ANN search. Facebook's foundation. The building block inside many vector DBs. Use directly when building a custom DB yourself. Benchmarking or embedding in a PyTorch pipeline with no ops. For anything production-facing, add a wrapper or use a managed DB.

`IndFlat, IVFFlat, HNSW, PQ | GPU support | Python bindings`

pgvector (PostgreSQL) — extension

A PostgreSQL extension. High-dimensional vectors stored alongside your relational data. ACID compliance. SQL joins between vectors and structured data. Neither ops team (existing Postgres) nor DBA (familiar tooling) complains. Sweet spot for teams under scale. Neon's Serverless Postgres offers vector scale. Neon's Serverless Postgres offers vector scale.

`CREATE EXTENSION vector | Supports: Hnsw, IVFFlat support`

Azure AI Search — managed cloud

The enterprise Azure choice. Native hybrid search (BM25 + vector + semantic ranker). Integrated with Azure OpenAI embeddings. Managed chunking + embedding pipeline. Role-based access control at index level. Managed identity auth. Deploy in your Azure tenant. Compliance inherited from Azure. Best if your stack is Azure-native. Azure Functions triggers and Azure Document Intelligence.

`Hybrid (BM25 + vector) | Semantic ranking | Managed chunking + embedding pipeline`

Milvus — open-source + managed

Purpose-built for massive scale. Distributed architecture. Supports billion-scale vector collections. Multi-modal vectors. Multiple index types (HNSW, IVF, DiskANN). Overkill for most RAG systems — exists at large scale. Zilliz Cloud is the managed version. Enterprise search. Zilliz Cloud is the managed version.

`Milvus | Zilliz Cloud | Billion-scale vector collections | Multi-modal vectors`

Two more that matter: LancoDB uses Lance columnar format on disk (zero-copy reads, very fast for ML workloads, right Arrow integration, excellent for local and embedded use). Redis Vector Search adds vector indexing to an existing Redis cluster — great if you're already using Redis for caching and want to colocate semantic cache with your operational cache.

2. ANN Algorithms — The Engine Room

The core problem with 10 million vectors at 1536 dimensions, an exact brute-force nearest neighbor search requires computing 10 million dot products per query. At ~1ms per 1000 dot products, that's 10 seconds per query. Completely unacceptable. ANN algorithms make a tiny accuracy loss for 1000× speed gains.

Here's the conceptual map of all major ANN approaches:

ANN algorithm families

ANN algorithms

Graph-based
HNSW, NSG

Clustering-based
IVF, IVFPQ, ScaNN

Hash/Tree-based
LSH, KD-tree, Annoy

Algorithm	Build time	Query speed	Memory	Best for
HNSW	Slow	Very fast	High (RAM)	Production RAG
IVFFlat	Fast	Medium	Low	Low-memory
IVFPQ	Fast	Medium	Very low	Billion vectors
LSH	Very fast	Slow	Low	Streaming/online
ScaNN (Google)	Slow	Fastest	Medium	Google scale
Flat (brute)	None	Slow	None	<50K vectors

3. HNSW Deep Dive — The Algorithm That Powers Production RAG

HNSW (Hierarchical Navigable Small World) is the dominant ANN algorithm in production vector databases. It's what Qdrant, Weaviate, Pinecone, pgvector, and most other databases use under the hood. Understanding it deeply is a genuine differentiator in interviews.

The core intuition: HNSW builds a multi-layer graph where each layer is a "navigable small world" — a graph where you can get from any node to any other node in few hops. The top layers are sparse (long-range connections for fast coarse navigation). The bottom layers are dense (short-range connections for precise local search).

Think of it like navigating a city: you start on a highway (top layer, fast, coarse), exit to a main road (middle layer), then navigate local streets to your exact destination (bottom layer, precise).

M (connections/node)
16

ef_construction
200

ef_search
64

M 4 64
ef_construction 100 500
ef_search 10 200

Recall@10 (sift 1m)
78.2%

Index memory (sift 1M dim 128)
6.3 GB

Query speed (sift)
~11.1 ms

Layer 2 (top) — sparse, long range
~5% of nodes. Entry point for every query. Broad jumps across the entire vector space. Very few edges per node. Fast to traverse. Gets you to the right region of 2-3 hops.

Layer 1 (mid) — intermediate connectivity
~30% of nodes. Narrows to the local neighborhood. M connections per node. Bridges between the coarse top layer and the precise bottom layer.

Layer 0 (bottom) — all nodes, 2M connections
100% of nodes. Every vector here. Each node has up to 2×M connections. Greedy search walks the denser part of the graph to find the true nearest neighbors. ef_search controls how many candidates are evaluated at this layer.

Configuration looks reasonable. Tune ef_search at query time for recall/latency balance without rebuilding the index.

The three HNSW parameters you must know cold:

`M` is the number of bidirectional connections per node. Higher M = better recall but more memory (proportional). Default 16 is a good starting point. Values 8–32 cover 95% of use cases. At M=64 you get marginal recall gains with significant memory cost.

`ef_construction` controls how many candidates are evaluated when inserting each new node during index construction. Higher = better graph quality = better recall at query time. But it only affects build time, not query time. Default 200, production recommended >200. Never lower than 100 for production.

`ef_search` is the only parameter you can tune at query time without rebuilding. Controls the size of the dynamic candidate list during search. Higher = better recall but slower queries. The key knob: you can express `ef_search` as a query time parameter to let different use cases optimize their own recall/latency trade-off.

4. IVF, PQ, and Quantization Strategies

While HNSW dominates for moderate-scale RAG (millions of vectors), at billion-vector scale you need quantization — techniques that compress vectors to reduce memory and speed up computation.

IVF (Inverted File Index): Cluster all vectors into `nlist` Voronoi cells using k-means. Store a posting list for each cluster. At query time, embed the query, find its nearest `nprobe` clusters, and search only within those clusters. The `nlist`/`nprobe` ratio is the recall/speed tradeoff. `nprobe=1` is fastest but lowest recall. `nprobe=nlist` expands brute force (full recall, no speedup).

Product Quantization (PQ): Splits each high-dimensional vector into M equal length sub-vectors. Trains a codebook of K=256 centroids for each sub-vector space. Represents each sub-vector as a single byte (or bits). Compresses a 1536-dim float32 vector (6144 bytes) into M bytes (e.g. 96 bytes for M=96 sub-vectors — 64× compression). Distance calculation becomes fast table lookups.

IVFPQ combines both. IVF for coarse candidate selection, PQ for compressed vector storage within each cluster. This is how Billion-scale vector search is feasible.

Scalar Quantization (SQ): Simpler than PQ. Converts float32 values to int8. 4× compression, ~1% recall loss. Supported natively by Qdrant, Weaviate, and Milvus. Most production systems should use SQ by default — the compression/quality ratio is excellent.

Binary Quantization: Each float32 → 1 bit based on sign. 32× compression, ~5-9% recall loss. Use for first-stage retrieval only, then re-score top-N with full vectors. Qdrant's binary quantization is particularly well-implemented.

5. Metadata Filtering — The Hidden Complexity

This is where most RAG engineers get surprised by production failures. Filtering sounds simple: "give me chunks from source=policy_v2 where date > 2023". In practice, it interacts with the ANN index in ways that can silently destroy recall.

Post-filtering (naive)

ANN search (all vectors)

Return top 10

Filter by metadata

Return 0-3 results

Problem: ANN found 10 neighbors in wrong partition. Filter removes most of them.

Pre-filtering (correct)

Build filtered candidate set

ANN search within subset

Return top-k (filtered)

ANN only searches vectors that satisfy the filter. Recall preserved.

Qdrant: uses payload index + Roaring bitmap graph search

The filtering challenge in depth:

Post-filtering is the naive approach: run ANN, get top-k, filter by metadata. The problem is severe when filters are selective. If only 3% of your corpus passes the filter, and you retrieve top-10, statistically most of those 10 vectors are from the 97% that will be filtered out. You end up with 0-1 results. Increasing k to compensate (retrieve top-200 then filter) defeats the purpose of ANN.

Pre-filtering requires the vector DB to integrate metadata filters with the ANN search. Qdrant does this exceptionally well — it builds a payload index and modifies the HNSW graph traversal to only visit nodes whose payload satisfies the filter. Weaviate uses a similar approach with inverted indexes. Pinecone uses namespaces and metadata filtering at the index level.

Hybrid filtering is the practical solution for high-selectivity filters when the filtered subset is small enough, full back to brute-force search on just those vectors. Qdrant calls this "exact search mode" and triggers it automatically when the payload filter reduces candidates below a threshold.

Metadata schema design matters:

```
python

# Rich schema - so index on frequently-filtered fields
{"text": "...", "metadata": {"source": "policy_v2", "date": "2024-01-15",
 "section": "coverage"}}

# Production schema - structured, indexed, filterable
{
  "id": "chunk_abc123",
  "tenant_id": "geisinger_client_42",          # for multi-tenancy isolation
  "doc_id": "policy_2024_q1_2024.pdf",         # for sequence filter
  "doc_version": "v2",
  "page_num": 12,
  "section_type": "coverage",                  # enum - low cardinality
  "created_at": 1704067200,                    # unix timestamp - range filter
  "carrier": "healthx",
  "plan_id": "plan_a"
}
```

Index every field you'll filter on. Don't put nested dicts in metadata — flatten them. Use low-cardinality enums for categorical fields (faster to index and filter than free-text).

Namespaces vs collection management:

Namespaces/partitions are for multi-tenancy — one logical index split by tenant ID. Each query is scoped to a tenant's namespace. Pinecone, Qdrant, and Weaviate all support this. Collections are separate indexes — use when tenants need completely different schemas or embedding models.

6. Indexing Strategies — Choosing the Right Index Type

Flat (brute force) | use for < 10K vectors, evaluation ground truth

Computes exact distance to every vector. Zero approximation error. No index to build. Use for: eval datasets (ground truth comparison), tiny corpora where speed doesn't matter, validating that ANN recall is acceptable before switching. FAISS IndexFlatL2 / IndexFlatIP are the canonical implementations.

HNSW | use for 10K–100M vectors, latency-sensitive production RAG

The production default. Best recall/latency ratio in its class. Extra graph must fit in RAM — the primary constraint. At M = 16(default) ~10GB RAM. At M=64 vectors require memory-mapping or multi-node. Qdrant, Weaviate, Pinecone, pgvector all support HNSW. Qdrant's implementation is the most well-optimized.

IVFPQ | use for 50M+ 8K vectors, memory-constrained

IVF clustering + Product Quantization compression. A 1536-dim float32 vector (6144 bytes) compressed to ~96 bytes (64×). Enables billion-scale search on a single machine. Recall lower than HNSW (~85-90% typical). Training phase required. Best when memory is the binding constraint. Good for recommendation systems and large scale enterprise search.

Scalar quantization (SQ8) | use for most production RAG, cost optimization

float32 → int8. 4× memory reduction. ~1% recall loss. Supported natively: Supported natively in Qdrant and Weaviate. Should be the default for production. Qdrant's implementation — the memory savings offset cost of RAM. Pairs well with HNSW. Qdrant's implementation — the memory savings offset cost of RAM. Pairs well with HNSW which actually improves overall performance. Qdrant: quantization_config.scalar.quantile=0.99 Weaviate: PQ enabled=true

On-disk indexing | use for very large corpora where RAM is the bottleneck

Stores the index on SSD instead of RAM. Slower than in-memory (1-10ms vs 0.1ms in memory) but enables much larger corpora. Qdrant's on-disk indexing is the most mature. Use NVMe SSDs. + SATA will be too slow. Cache frequently accessed nodes in RAM using mmap. Good strategy: put HNSW graph on disk, keep quantized vectors in RAM for distance computations.

Streaming ingestion | use for continuously updating corpora

HNSW supports online insertion — new vectors are added to the existing graph without full rebuild. This is critical for production systems where documents are updated continuously. Kafka + Qdrant pipelines document change → embed → upsert to vector DB. Key concern: graph quality degrades slowly over time (suboptimal connections accumulate). Batch inserts in windows of 100-1000 vectors. Use composite primary keys for deduplication/upsert semantics.

7. DB Selection Framework — How to Choose

Answer four questions to reach a recommendation.

What's your deployment preference?

Fully managed (no ops) | Azure stack | Self-hosted | Existing Postgres | Prototype / local

Interview Question Bank — Domain 4

Basics **What is approximate nearest neighbor search and why do we need it?**

Exact nearest neighbor search requires computing the distance vector to every vector in the database — O(n×d) per query. At 10M vectors × 1536 dims, that's billions of floating point operations per query, taking seconds. ANN makes it sub-millisecond by trading a tiny fraction of accuracy (typically >95 recall loss for 100-1000× speedup) by using specialized data structures (graphs, clusters, hash tables) that skip vectors unlikely to be nearest neighbors. In production RAG, ANN retrieves top-50 candidates in ~10ms at billion-vector scale. The "approximate" part means you get the 10 nearest neighbors with very high probability (~99%), not a guaranteed perfect result.

Basics **Explain HNSW at a conceptual level. Why is it the dominant algorithm?**

HNSW builds a multi-layer graph. The layers have sparse connections (few nodes, long-range edges) — these enable fast coarse navigation across the entire vector space. The bottom layer (layer 0) has every vector with 2M connections — this is where the fine-grained search happens. A query starts at the entry point in the top layer, greedily hops toward the query vector, drops to the next layer when a local minimum is found, and repeats until reaching layer 0 where it collects the final nearest neighbors. Hierarchical structure because: O(number) recall vs speed trade-off: [2] supports online insertion without full rebuild. [3] works well with high-dimensional embeddings. Production default: [4] the ef_search parameter lets you tune recall vs speed at query time without rebuilding.

Mid-level **What are M, ef_construction, and ef_search in HNSW? How do you tune them?**

M controls the number of bidirectional connections per node (higher = more memory, more accurate). ef_construction controls how many candidates are evaluated when inserting each node (higher = better graph quality, slower build). Default 200, never lower than 100. ef_search controls the candidate list size when searching (higher = better recall, slower query). Tuning strategy: fix M at 16, set ef_construction to 200, then tune ef_search to your recall target using a sample of real queries.

Mid-level **Why does metadata filtering break ANN recall, and how do you fix it?**

ANN is optimized to find nearest neighbors in the full vector space. When you filter by metadata post-search, you retrieve k vectors that discard those that fail the filter. If only 5% of the corpus and you retrieve top-10, you get 0-1 results on average. Fix: pre-filtering (apply metadata filter before ANN search) or post-filtering with over-retrieval (retrieve k/selectivity candidates, then filter). Pre-filtering is cleaner but requires database support. Qdrant's payload index integrates the metadata filter into the graph traversal, only visiting nodes whose payload satisfies the filter. Qdrant does this with a payload index + modified HNSW search. Weaviate uses inverted indexes. When the filtered subset is small (high selectivity), Qdrant switches to exact search (brute force) on the filtered subset. Good vector DBs do this automatically.

Mid-level **What is Product Quantization and when would you use it over HNSW?**

PQ splits each vector into M sub-vectors and represents each with a single byte (index into a 256-centroid codebook). A 1536-dim float32 vector (6144 bytes) becomes ~96 bytes — 32-64× compression. Distance calculation becomes table lookups instead of full dot products. Use when: (1) memory is the binding constraint (HNSW graph must fit RAM) (2) slight recall loss (~85-90% recall) is acceptable (3) billion-scale vectors. Typically combined with IVF: IVFPQ = IVF clustering + PQ compression. Qdrant supports binary quantization (recommendation engines, semantic deduplication at scale). For typical RAG systems (1M-10M chunks), HNSW with scalar quantization is better — better recall, simpler to operate, and the memory cost is manageable.

Senior **Design a vector DB architecture for a multi-tenant SaaS RAG system with 500 enterprise clients, each with 100K-10M documents.**

Key decisions: (1) isolation model — namespaces within a shared collection vs. separate collections per tenant — namespaces for small tenants, separate collections for large tenants (2) HNSW parameters — M=16 for large tenants (cost-sensitive), M=32 for premium tenants (quality-sensitive) (3) quantization — SQ8 for all tenants (4) hot/cold tiering — keep active tenants' indexes in RAM, lazy-load inactive tenants from SSD (5) ingestion pipeline — Kafka topic per tenant, batch upsert, dedup by content_hash (6) monitoring — per-tenant recall@10 on a golden eval set, p95 latency, index memory usage. Critical: tenant_id must be a mandatory filter on every query, enforced at the API layer, not trusted to the client.

Senior **Pinecone vs Qdrant vs pgvector — how do you choose?**

Choose Pinecone when: team is small, zero ops budget, need to ship in a day, comfortable with SaaS pricing, don't need self-hosted. Choose Qdrant when: best in-class metadata filtering is required (Qdrant's filtered HNSW is the most sophisticated), need hybrid search (sparse + dense vectors), cost-sensitive at scale (Qdrant Cloud or self-hosted), building RAG+agentic memory. Choose pgvector when: already on Postgres and <10M vectors — one infrastructure, SQL joins between vectors and relational tables, same backup/monitoring/ops as existing DB. The hidden cost of pgvector: it doesn't perform as well as Qdrant for heavy filtering workloads at scale. Migration path is painful. If you're at the decision point for a new production system and expect significant growth, go Qdrant directly.

The Mental Models That Win These Interviews

A vector DB is just a CPU for meaning spaces. The ANN index is the road network that lets you navigate to the nearest neighbors fast. HNSW builds a highway system — broad roads on top, local streets at the bottom — and you navigate from highway to local street until you arrive.

ef_search is the only knob you can turn in production without rebuilding. Everything else (M, ef_construction, index type, quantization) is baked into the index at build time. ef_search lets you answer: "for this query, is low latency or high recall more important?" Different use cases in the same system have different values.

Quantization = lossy compression for vectors. SQ8 is JPEG compression — perceptually identical but 4× smaller. Is it more like MP3? — the details are gone, but the overall signal is preserved. Use SQ8 by default in production; it's almost free quality loss for a massive memory saving.

Metadata filtering + ANN is harder than it looks. Post-filtering silently destroys recall. Pre-filtering requires vector DB support. When someone says "add a filter for source document", the right engineering response is "I need to verify the vector DB's filtering architecture before promising any recall guarantees."

Multi-tenancy strategy is an architectural decision, not a config. Namespace + soft isolation (shared index, tenant_id filter) = cheap, fast, operationally simple. Separate collections/indexes = hard isolation (separate HNSW graphs, full isolation but more ops). Separate DB = full isolation with data phase separation (the highly regulated tenants). Match your tier. ⚠ a strategy to your compliance requirements.

Reply...

Sonnet 4.6

Claude is AI and can make mistakes. Please double-check responses.